Sentinel-RX: Security-Aware Agentic Recovery of Failing Systems

Technical brief — Humain Hackathon, Challenge 1: Agentic Recovery of a Failing System

Abdulrahman Alharbi · 30 October 2026

Companion to the full technical report, `sentinel_rx_technical_report.pdf` in the same folder.

In one paragraph. Sentinel-RX is an agent that identifies, diagnoses, recovers and *verifies* the recovery of a failing enterprise system when several faults are active at once and a security compromise may be hiding inside the operational incident. Deterministic detectors compress telemetry into ranked evidence, a local 7B model reasons over it in a budgeted loop, a safety layer keeps the actions harmless, and a verification gate refuses to say "recovered" until the telemetry agrees. On 20 held-out incidents it recovers 100 % of systems with 0 % false claims and zero harmful actions, where a rule-based runbook recovers 45 % and a single-shot LLM falsely claims success 70 % of the time. Everything runs on one laptop, free, with no cloud and no Docker, and every number replays from a committed cache without a model installed.

1 Why this matters

The operational problem. Incident response is the most expensive, least automated part of running software. Today's auto-remediation is a runbook: if a threshold fires, run an action. Runbooks cannot tell a symptom from its cause in a dependency graph, cannot handle two faults at once, and cannot tell whether what they did worked. LLM agents promise to close that gap, but the published benchmarks inject one accidental fault per scenario and the best agents resolve roughly 11 % of realistic SRE tasks [1, 2].

The security problem. Some "performance incidents" are intrusions. A compromised host phoning home to a command-and-control server looks, to an ops tool, like a slightly slow service. If an agent restarts it, the forensic evidence is gone and the implant comes back. No AIOps benchmark tests whether an agent recognises this or orders its actions correctly (contain, then eradicate). Sentinel-RX does both, and the benchmark measures it.

The trust problem. An agent that says "recovered" is believed, and the incident is closed. A false claim is worse than no agent. Sentinel-RX treats its own claim as something to be verified against telemetry, and the benchmark reports false recovery as a headline metric.

What is useful about it.

- **A design that degrades safely.** Honesty and harmlessness live in the detectors, the safety layer and the verifier, not in the model. Swapping the 7B model for a 1.5B one halves recovery but keeps 0 % false claims and zero harmful actions: a weaker planner becomes an honest escalator, not a confident liar.
- **A benchmark others can reuse.** Multi-fault scenarios with disjoint held-out combinations, hidden security faults, perturbations, sanity-floor baselines, ablations, bootstrap CIs, a split guard that refuses to run held-out without `-final`, and a replay cache. It is small, but the *design* transfers to any environment.
- **Cheap.** 6 103 tokens and 4.6 model calls per incident on a laptop-hosted model; the same loop reading raw telemetry needs 5.7× more tokens to recover 2.9× fewer incidents.

- **Real processes, same code.** The agent that scores the benchmark also recovers a live three-fault incident on five real local processes, byte-identical.

2 How it maps to the judging criteria

criterion	evidence (every number is from `results/`, held-out split unless stated)
Task performance	20 held-out scenarios never used in development, 3 seeds ($n = 60$): top-1 root cause 92 % [83, 98], per-fault recall 82 %, recovered 100 % [100, 100], false "recovered" 0 %, harmful actions 0.00 per run, action precision 81 %. Runbook: 45 % recovered. Single-shot LLM: 30 % recovered, 70 % false claims. Recovery is flat at 100 % from one to three concurrent faults (table 1).
Robustness & adaptation	Recovery stays 95 %–100 % under noisy logs, missing metrics, renamed services and timestamp jitter; an *unseen* fault type is still recovered 100 % (top-1 62 %); a 1.5B model keeps 0 % false claims and 0 harm. Same agent runs unchanged on the simulator and on five real processes (section 4).
Evaluation & iteration	Four dev iterations with before/after numbers (top-1 70 → 90 %, recovered 73 → 100 %, calls 7.4 → 4.0); a mid-development *scorer audit* that found two lenient spots, fixed them and re-scored every earlier run; do-nothing and random sanity floors (0 % recovered); four ablations that locate each guarantee in one component; 108 tests; CI with a replay smoke run and regression floors (section 5).
Technical judgment & creativity	Hybrid design (detectors compress, LLM integrates) measured against its raw-telemetry twin; a RandomForest beacon detector on scale-invariant timing features with destination knowledge as a prior (AP 0.999, 0 false positives on health checks and update polls); contain-before-restart ordering; transactional rollback; a verification gate with a remembered healthy baseline and a known-beacon rule; health = $\frac{1}{2}\bar{h} + \frac{1}{2}\min h$ so one sick service cannot hide (section 3).
Engineering quality	One `make` target per step; pinned dependencies; typed, ruff-clean modules under 400 lines; one `Env` interface for simulator and live system; content-addressed LLM cache so `make smoke` and CI need no model; one git branch per phase merged with `-no-ff`; a run log with git revision for every held-out run (section 6).
Communication	README with real console output, a static site with leaderboard, charts and an incident replay viewer, DESIGN / RESULTS / LIMITATIONS documents, a literature review with a gap analysis the design answers point by point, the full technical report, and this brief.

3 What it is

Figure 1 shows the system. The agent talks to an environment only through a small interface (status, metrics, logs, flows, eight allow-listed actions); it never sees the simulator's fault list. Four deterministic detectors turn thousands of records into a short ranked evidence list: robust z-scores against a remembered healthy baseline, log-template signatures, a dependency-aware ranker that discounts upstream symptoms explained by a sick dependency (the single biggest win over threshold runbooks), and a RandomForest beacon detector on scale-invariant inter-arrival features. The LLM (`qwen2.5:7b` via Ollama, two JSON calls per iteration) proposes ranked hypotheses and up to three targeted checks, then an ordered plan. The safety layer enforces the allowlist, refuses containment of hosts without beacon evidence, reorders plans so containment precedes restarts, and undoes any action that drops observable health by more than 0.15. The verification gate compares post-action telemetry with the healthy baseline and re-scores known beacon pairs; the agent may claim "recovered" only when nothing is flagged, otherwise it loops (budget 8) or escalates with the trace.

Design decision — why hybrid, why a 7B model, why no Docker

The LLM is good at integrating heterogeneous evidence and ordering a plan, bad at scanning four thousand flow records. Giving it ranked evidence instead of raw telemetry is both the cheapest and the most accurate configuration tested (table 1). A locally hosted 7B model keeps the whole system free and private, and its weaknesses (mixing facts between services, ignoring status fields) were handled with per-service evidence grouping and a repeated-plan guard rather than a bigger model. Five stdlib processes boot in a second and need no container runtime, which the brief forbade anyway.

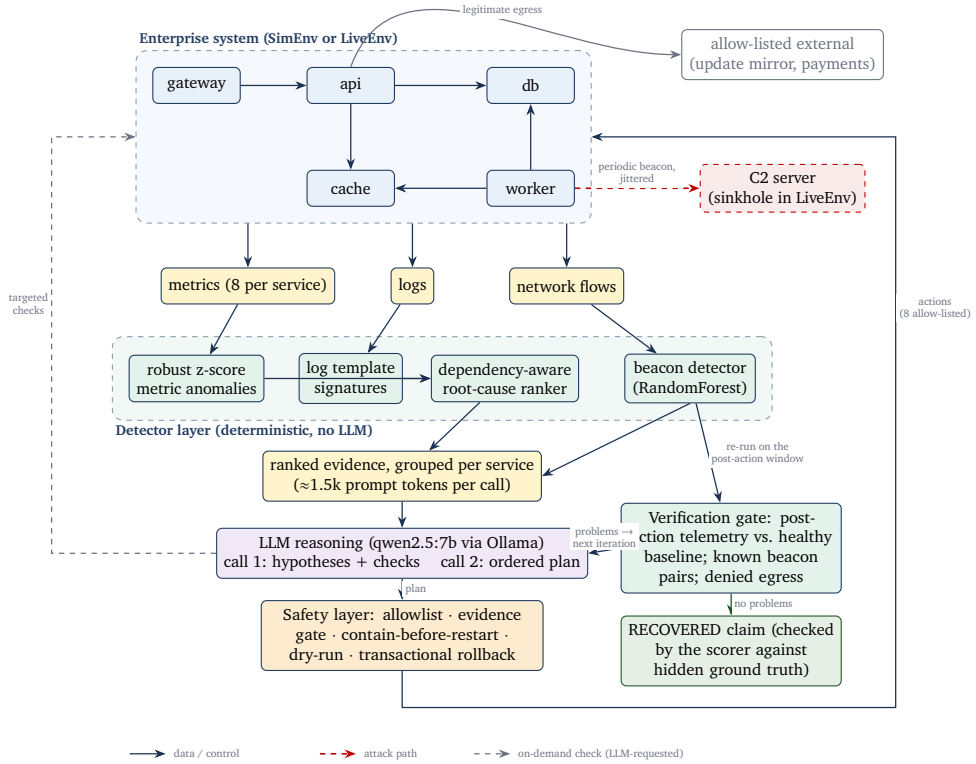


Figure 1: System architecture. Telemetry from the enterprise system is compressed by four deterministic detectors into ranked evidence; the LLM reasons, the safety layer screens and executes, and the verification gate closes the loop. The beacon path from `worker` to the C2 server is the hidden security incident.

4 Results

Table 1: Held-out results (`qwen2.5:7b`, 20 scenarios, 95 % bootstrap CIs; n = runs). Harmful actions are counts per run. Source: `results/summary.md`.

config	top-1	recall	recovered	false "rec."	harmful / run	action prec.	calls	tokens	n
agent	**92** [83, 98]	**82** [74, 89]	**100** [100, 100]	**0** [0, 0]	**0.00**	**81** [74, 88]	4.6	6 103	60
runbook	87 [78, 95]	81 [74, 87]	45 [32, 58]	5 [0, 12]	0.00	74 [68, 80]	0	0	60
single_shot	63 [50, 75]	50 [41, 59]	30 [20, 42]	70 [58, 80]	0.27 [0.15, 0.38]	37 [30, 43]	1.0	2 421	60
llm_only	35 [15, 55]	44 [28, 61]	35 [15, 55]	15 [0, 30]	0.55 [0.15, 1.05]	25 [13, 39]	18.4	35 018	20
do_nothing	0	0	0	0	0.00	n/a	0	0	60
random	0	0	0	10 [3, 18]	3.45 [2.92, 3.98]	5 [4, 7]	0	0	60

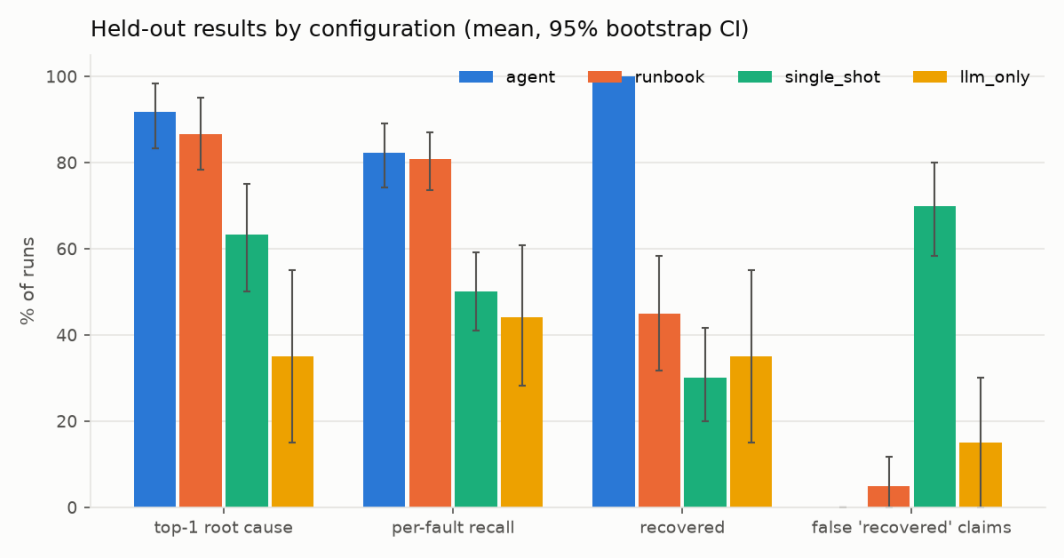


Figure 2: Held-out top-1, per-fault recall, recovery and false-recovery rate per configuration, 95 % bootstrap CIs.

Concurrency. Recovered / top-1 by number of concurrent faults: agent 100 %/75 %, 100 %/93 %, 100 %/100 % for 1, 2, 3 faults; runbook 75 %/50 %, 37 %/93 %, 38 %/100 %; single_shot 75 %/50 %, 33 %/67 %, 0 %/67 %. Every baseline degrades with concurrency; the agent does not. Its single-fault top-1 misses are type confusions where the fix coincides (a cache latency injection labelled `cache_failure`, resolved by a restart), counted as diagnosis misses but not recovery misses.

Security. In the 27 held-out runs containing a C2 beacon the agent recovered 100 %, never restarted a compromised host before containing it and never contained an uncompromised one. single_shot and llm_only did so routinely (0.48 and 1.11 harmful actions per run); a random walk over the same eight actions is harmful 3.04 times per run. The action space is dangerous by default; the safety layer, not the model, makes it safe.

Table 2: Robustness (agent, held-out variants, 1 seed) and ablations (held-out, 1 seed, $n = 20$; deltas vs. the full agent).

variant	top-1	recovered	false "rec."	harmful / run
full agent, clean held-out (3 seeds, $n = 60$)	92 %	100 %	0 %	0.00
noisy logs (+80 % benign lines)	95 %	100 %	0 %	0.00
missing metrics	90 %	95 %	0 %	0.00
renamed services	90 %	100 %	0 %	0.00
timestamp jitter (±2 s)	90 %	100 %	0 %	0.00
unseen fault type (dns_failure, $n = 8$)	62 %	100 %	0 %	0.00
smaller model (qwen2.5:1.5b)	40 %	50 %	0 %	0.00
ablation: no verification gate	80 % (−12)	20 % (−80)	80 % (+80)	0.00
ablation: no loop (one iteration)	80 % (−12)	20 % (−80)	0 %	0.00
ablation: no beacon detector	85 % (−7)	50 % (−50)	35 % (+35)	0.05
ablation: no safety layer	90 % (−2)	100 % (0)	0 %	0.60 (+0.60)

Key finding — each component protects a different metric

Without the verification gate the agent takes the same actions but *claims* success in 80 % of runs; without the loop it recovers only 20 % but escalates honestly; without the beacon detector recovery halves and no security scenario is recovered; without the safety layer recovery is unchanged but 0.60 harmful actions per run appear. Diagnosis comes mostly from

the detectors (the LLM-free runbook reaches top-1 87 %); recovery from the loop; honesty from the gate; harmlessness from the safety layer. The model's contribution is integration and planning over evidence it did not have to find.

Cost. Agent: 4.6 calls, 6 103 tokens, 2.2 iterations and 190 s of laptop time per incident. llm_only: 18.4 calls, 35 018 tokens, 816 s. The 1.5B model is not cheaper either (17.8 calls, 29 659 tokens) because it loops on wrong plans.

Live demonstration. Figure 3 is the recorded `make demo-live` run on five real processes: a memory leak on `api`, a connection-pool hold on `db` and a C2 beacon on `worker` every ≈4 s, injected together after a 35 s baseline (health 1.00 → 0.10). The model's first plan tried to restart `worker`; the safety layer blocked it because the beacon detector had flagged the host. Verification refused "recovered" twice, naming the live beacon. In the third iteration the model blocked `worker`'s egress and only then restarted it. Ground truth from the live system afterwards: no active faults, restart-before-containment = 0, final health 1.00; 6 LLM calls, 9 058 tokens.

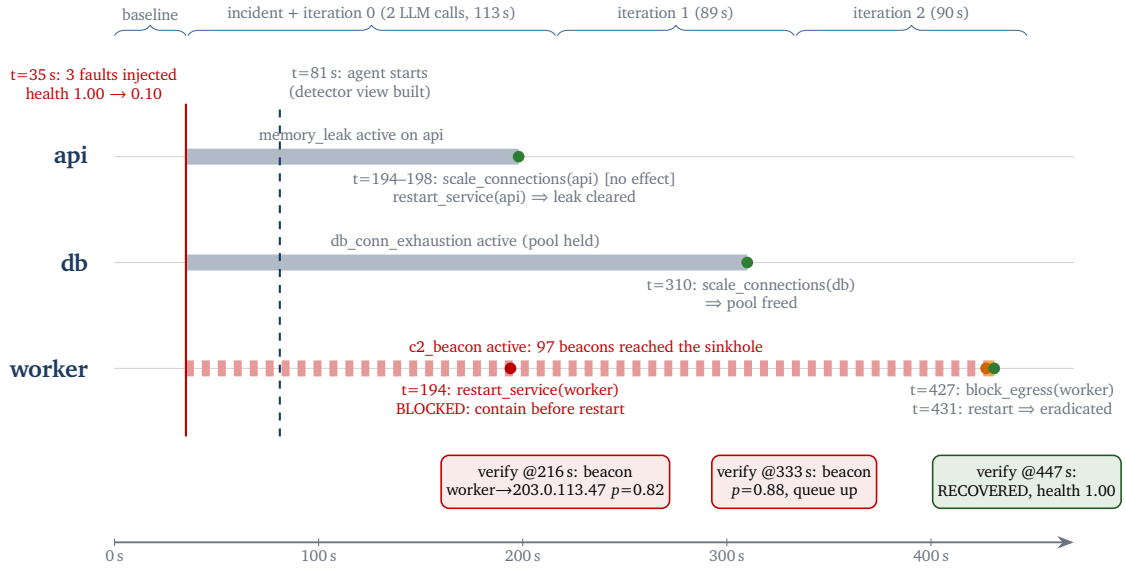


Figure 3: The recorded live-demo incident: memory leak on `api`, pool exhaustion on `db` and a C2 beacon on `worker`, on five real processes. Bars are the hidden fault lifetimes; dots are the actions that ended them; boxes below the axis are the verifier's verdicts. Times are seconds since the services started (`results/demo_live_trace.json`).

5 How the numbers were kept honest

- **Split discipline.** 30 dev scenarios for tuning, 20 held-out with fault combinations and type multisets never seen in dev and a disjoint seed range; the runner refuses non-dev splits without `-final` and logs every such run with its git revision. The agent, detectors, prompts, simulator and scorer were frozen (tag `v1-frozen`) before the first held-out run.
- **Sanity floors.** do_nothing and random are permanent baselines: any scenario they "recover" is a broken check. They recover 0 % on held-out and 0 %–1 % on dev.
- **Scorer audit.** Mid-development the recovery metric was challenged. Tests now prove that no fault clears on its own in 300 ticks and none is resolved by any action on the wrong target or any wrong action on the right target ($8 \times 5 \times 10$). Two lenient spots were found and fixed: the observable health formula let one sick service hide behind four healthy ones (now $\frac{1}{2}\bar{h} + \frac{1}{2}\min h$), and containment of an uncompromised host was not counted as harmful. Re-scoring showed that agent v0 and v1 were *not* harmless (0.27 and 0.23 wrong containments per run); the

containment gate introduced in v2 removed that. Pre-audit numbers are kept, labelled, in `docs/RESULTS.md`.

- **Iteration log.** v0 → v3 on dev: top-1 70 → 90 %, recovered 73 → 100 %, false claims 0 % throughout, calls 7.4 → 4.0, each change named with its before/after numbers.
- **Regression checks.** 108 tests (simulator, faults, scenarios, detectors, agent with a scripted LLM, scorer, runner, live environment); CI runs ruff, the tests, a replay smoke run and floors (recovered ≥ 0.8, top-1 ≥ 0.8, false recovery = 0, harmful = 0).

6 Reproduce it

Python 3.12, no Docker. Replay targets need no model; recording targets need Ollama with `qwen2.5:7b`. Every number in this brief replays from the committed cache.

```
make setup           # venv, pinned deps, scenarios, beacon detector training (~1 min)
make test            # 108 tests, no LLM needed
make smoke           # 5 dev scenarios from the replay cache, no LLM (< 60 s)
make demo-live       # 5 real local processes + 3 concurrent faults + the agent (needs Ollama)
make all-final       # held-out + perturbations + ablations + report (needs Ollama; hours)
python -m bench.runner --split heldout --config agent --seeds 3 --final --replay   # exact replay
python -m http.server -d site 8000     # leaderboard, charts, incident replay viewer
```

7 Limitations, stated plainly

Limitation — what this does not show

The simulator is small and hand-written (five services, nine known fault types with scripted symptoms that match the taxonomy the agent is told), so the benchmark measures evidence integration and planning more than open-world discovery; the unseen fault type is the only exception. The beacon detector was trained on synthetic timings, not real captures. llm_only, perturbations, ablations and the small-model run use one seed (the 3-seed llm_only pass was stopped twice at 10/60 for time; both attempts are in the run log), so their deltas are directional. Replay is exact but re-recording with Ollama is not bit-stable. Rollback is best-effort (restarts and rollbacks are irreversible, and recorded as such). No multi-step attacks, network partitions or multi-cluster topologies. The GitHub remote, CI run and site have not been published yet; everything is local.

Path to production. Swap the telemetry adapters (NetFlow/Zeek, Prometheus, a log store) behind the same interface; retrain the beacon model on real captures; map the eight actions to existing runbook automations behind the same safety layer with human approval for irreversible ones; keep the verification gate as the only path to “resolved”; keep the replay cache as the audit log.

References

[1] S. Jha *et al.*, “ITBench: Evaluating AI agents across diverse real-world IT automation tasks,” in *Proc. ICML*, 2025.

[2] J. Xu *et al.*, “OpenRCA: Can large language models locate the root cause of software failures?” in *Proc. ICLR*, 2025.